

PRACTICAL SHEET – 5 (ARYAN KUSHWAHA – 245703428)

Q1: // Brute Force Pattern Matching with Comparison Counter

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

int totalComparisons = 0;

vector<int> bruteForceSearch(const string& T, const string& P) {
    vector<int> matchIndices;
    int n = T.length();
    int m = P.length();

    if (m == 0 || n < m) {
        return matchIndices;
    }

    for (int i = 0; i <= n - m; ++i) {
        int j = 0;
        while (j < m) {
            totalComparisons++;
            if (T[i + j] != P[j]) {
                break;
            }
            j++;
        }
        if (j == m) {
            matchIndices.push_back(i);
        }
    }

    return matchIndices;
}

int main() {
    string T = "ABABDABACDABABCABAB";
    string P = "ABABCABAB";

    totalComparisons = 0;

    vector<int> indices = bruteForceSearch(T, P);

    cout << "Text: " << T << "\n";
    cout << "Pattern: " << P << "\n";
    cout << "Match indices: ";
    if (indices.empty()) {
        cout << "None";
    } else {
        for (int i = 0; i < indices.size(); ++i) {
            cout << indices[i] << " ";
        }
    }
}
```

```

    }
}
cout << "\nTotal character comparisons: " << totalComparisons << "\n";

return 0;
}

```

Output:

```

Text: ABABDABACDABABCABAB
Pattern: ABABCABAB
Match indices: 10
Total character comparisons: 29

```

Q2: // LPS (Longest Prefix Suffix) Array Construction

```

#include <iostream>
#include <string>
#include <vector>
using namespace std;

vector<int> computeLPSArray(const string& P) {
    int m = P.length();
    vector<int> lps(m, 0);

    if (m == 0) {
        return lps;
    }

    int len = 0;
    int i = 1;

    while (i < m) {
        if (P[i] == P[len]) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) {
                len = lps[len - 1];
            } else {
                lps[i] = 0;
                i++;
            }
        }
    }

    return lps;
}

int main() {

```

```

string pattern1 = "AAAA";
string pattern2 = "ABCDE";
string pattern3 = "AABAACAABAA";
string pattern4 = "AAACAAAAAC";

vector<string> patterns = {pattern1, pattern2, pattern3, pattern4};

for (int p = 0; p < patterns.size(); ++p) {
    vector<int> lps = computeLPSArray(patterns[p]);

    cout << "Pattern: " << patterns[p] << "\nLPS: ";
    for (int i = 0; i < lps.size(); ++i) {
        cout << lps[i] << " ";
    }
    cout << "\n\n";
}

return 0;
}

```

Output:

```

Pattern: AAAA
LPS: 0 1 2 3

Pattern: ABCDE
LPS: 0 0 0 0 0

Pattern: AABAACAABAA
LPS: 0 1 0 1 2 0 1 2 3 4 5

Pattern: AAACAAAAAC
LPS: 0 1 2 0 1 2 3 3 3 4

```

Q1: // Write a program to implement KMP string pattern matching algorithm, output must include total number of comparisons made during execution.

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

int totalComparisons = 0;

vector<int> computeLPS(const string& P) {
    int m = P.length();
    vector<int> lps(m, 0);
    int len = 0;
    int i = 1;

    while (i < m) {
        totalComparisons++;
        if (P[i] == P[len]) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) {
                len = lps[len - 1];
            } else {
                lps[i] = 0;
                i++;
            }
        }
    }
    return lps;
}

vector<int> KMPSearch(const string& T, const string& P) {
    vector<int> matchIndices;
    int n = T.length();
    int m = P.length();

    if (m == 0 || n < m) {
        return matchIndices;
    }

    vector<int> lps = computeLPS(P);

    int i = 0;
    int j = 0;

    while (i < n) {
        totalComparisons++;
        if (T[i] == P[j]) {
            i++;
            j++;
            if (j == m) {
                matchIndices.push_back(i - j);
                j = 0;
            }
        } else {
            if (j != 0) {
                j = lps[j - 1];
            } else {
                i++;
            }
        }
    }
    return matchIndices;
}
```

```

        i++;
        j++;

        if (j == m) {
            matchIndices.push_back(i - j);
            j = lps[j - 1];
        }
    } else {
        if (j != 0) {
            j = lps[j - 1];
        } else {
            i++;
        }
    }
}

return matchIndices;
}

int main() {
    string T = "ABABDABACDABABCABAB";
    string P = "ABABCABAB";

    totalComparisons = 0;

    vector<int> indices = KMPSearch(T, P);

    cout << "Text: " << T << "\n";
    cout << "Pattern: " << P << "\n";

    cout << "Match indices: ";
    if (indices.empty()) {
        cout << "None";
    } else {
        for (int i = 0; i < indices.size(); ++i) {
            cout << indices[i] << " ";
        }
    }

    cout << "\nTotal character comparisons: " << totalComparisons << "\n";

    return 0;
}

```

Output:

```

Text: ABABDABACDABABCABAB
Pattern: ABABCABAB
Match indices: 10
Total character comparisons: 32

```